Towards partial fulfillment for Undergraduate Degree Level Programme
Bachelor of Technology in Computer Science and Engineering

A Project Evaluation Report on: Applied Deep Learning on PlantDoc Dataset

Prepared by :

Sr.	Admission No.	Student Name
1	U22CS088	Aniruddh Jain
2	U22CS091	Nikhil Tiwari
3	U22CS094	Saud Masaud Alam
4	U22CS103	Mahek Patel

Class: B.TECH, IV (Computer Science and Engineering) 7th Semester
Year: 2025-2026

Guided by: Dr. Sourajit Behera

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SARDAR VALLABHBHAI NATIONAL INSTITUTE OF TECHNOLOGY,
SURAT - 395007 (GUJARAT, INDIA)

Student Declaration

This is to certify that the work described in this project report has been actually carried out and implemented by our project team consisting of:

Sr.	Admission No.	Student Name
1	U22CS088	Aniruddh Jain
2	U22CS091	Nikhil Tiwari
3	U22CS094	Saud Masaud Alam
4	U22CS103	Mahek Patel

Neither the source code nor the content of the project report have been copied from any other source.

Signatures of the Students:

Sr.	Student Name	Signature
1	Aniruddh Jain	
2	Nikhil Tiwari	
3	Saud Masaud Alam	
4	Mahek Patel	

Certificate

This is to certify that the project report entitled **Applied Deep Learning on PlantDoc Dataset** is prepared and presented by the following students:

Sr.	Admission No.	Student Name
1	U22CS088	Aniruddh Jain
2	U22CS091	Nikhil Tiwari
3	U22CS094	Saud Masaud Alam
4	U22CS103	Mahek Patel

Students of Final Year, Bachelor of Technology in Computer Science and Engineering.

SIGNATURES:

Supervisor Jury Head of Department

Acknowledgement

We express sincere gratitude to our supervisor, **Dr. Sourajit Behera**, for his guidance and support throughout the project.

Thanks are also due to the jury members for their valuable feedback.

Abstract

This project evaluates deep learning models on the **PlantDoc dataset** for plant disease detection. Various Keras models were compared, including **Xception, VGG16/19, ResNet50V2, DenseNet121, EfficientNet, NASNet, ConvNeXt, and MobileNet**. Xception achieved the highest baseline accuracy. Fine-tuning improved performance to **58.90% test accuracy** with robust precision, recall, and F1-score. Quantization enabled deployment with reduced model size (INT8: 21.55 MB). The study provides a systematic methodology for model comparison, fine-tuning, and deployability for real-world plant disease detection.

Contents

1 Introduction **7**
1.1 Background . . . 7
1.2 Research Questions Raised . . . 7
1.3 Research Gaps & Motivation . . . 7
1.4 Objectives . . . 8
1.5 Our Contributions . . . 8
1.6 Organization of the Report . . . 8

2 Literature Survey and Theoretical Background **9**
2.1 Theoretical Background . . . 9
2.1.1 Plant Disease Analysis: Motivation . . . 9
2.1.2 Plant Disease Analysis Approaches . . . 9
2.2 Literature Survey . . . 10
2.2.1 Challenges in Plant Disease Detection . . . 11
2.2.2 Summary . . . 11

3 Proposed Approach and Algorithm **12**
3.1 Preliminaries . . . 12
3.2 Formulation of the Problem . . . 12
3.3 Proposed Algorithm . . . 12
3.4 Theoretical Analysis of the Algorithm . . . 14
3.5 Algorithm Workflow . . . 14
3.6 Summary . . . 14

4 Methodology, Implementation, Evaluation & Analysis **15**
4.1 Tools and Platforms Used . . . 15
4.2 Dataset Used . . . 15
4.3 Model Training and Testing Approach . . . 15
4.3.1 Initial Keras Model Evaluation by Team Members . . . 15
4.3.2 Fine-tuning Xception Model . . . 16
4.4 Metrics of Evaluation . . . 16
4.5 Flow of Implementation . . . 16
4.5.1 Individual Model Training Workflow . . . 16
4.5.2 Fine-tuned Xception Model Workflow . . . 16
4.6 Results . . . 17
4.6.1 Xception Model Summary . . . 17

4.6.2 Classification Report 17
4.6.3 Confusion Matrix 18
4.6.4 Discussion and Analysis 18

5 Conclusion and Future Scope **19**
5.1 Limitations of the Proposed Solution 19
5.2 Impact 19
5.3 Future Scope 20
5.4 Concluding Remarks 20

Chapter 1

Introduction

1.1 Background

Agriculture is one of the most critical sectors supporting human life, and plant diseases significantly affect crop yield and quality. Traditional disease detection methods rely heavily on manual inspection, which is time-consuming, error-prone, and requires expert knowledge. With the advent of deep learning, automated plant disease detection has become feasible and efficient. Convolutional Neural Networks (CNNs) provide scalable solutions by learning hierarchical features directly from images, eliminating the need for manual feature engineering. Applying deep learning to datasets such as PlantDoc enables accurate, real-time detection of plant diseases across multiple species.

1.2 Research Questions Raised

To systematically study plant disease detection using deep learning, the following questions guide our research:

1. Which Keras CNN model performs best on the PlantDoc dataset in terms of accuracy and computational efficiency?
2. How does fine-tuning the Xception model impact performance on this dataset?
3. What is the effect of hyperparameter tuning and model quantization on accuracy, size, and deployability?
4. Can the trained models generalize effectively to unseen plant images with varying conditions such as lighting, background, and resolution?

1.3 Research Gaps & Motivation

Although deep learning has been widely applied for plant disease detection, most studies focus on a single model or dataset. There is limited research systematically comparing multiple state-of-the-art Keras models, evaluating not only accuracy but also efficiency, interpretability, and deployability. Moreover, quantization and model size considerations for edge deployment are often overlooked. This motivates our comprehensive evaluation of various CNN models and fine-tuning strategies to address these gaps.

1.4 Objectives

The main objectives of this project are:

1. To compare multiple pre-trained CNN models on the PlantDoc dataset.
2. To fine-tune the Xception model for improved performance and robustness.
3. To evaluate models using metrics such as accuracy, precision, recall, and F1-score.
4. To analyze model size, quantization effects, and deployment feasibility.
5. To provide insights and guidance for practical applications in precision agriculture.

1.5 Our Contributions

This project contributes to the field of automated plant disease detection by:

1. Performing a comprehensive evaluation of multiple pre-trained CNN models on the PlantDoc dataset.
2. Implementing fine-tuning strategies on Xception to improve test accuracy and class-wise performance.
3. Analyzing model size, computational requirements, and the impact of quantization for edge deployment.
4. Providing benchmarks, visualizations, and insights to guide future research and practical implementation.

1.6 Organization of the Report

The report is structured as follows:

- **Chapter 2:** Literature survey and theoretical background.
- **Chapter 3:** Proposed approach, problem formulation, and algorithm.
- **Chapter 4:** Methodology, implementation, evaluation, and analysis.
- **Chapter 5:** Conclusion, limitations, and future scope.

Chapter 2

Literature Survey and Theoretical Background

2.1 Theoretical Background

2.1.1 Plant Disease Analysis: Motivation

Automated plant disease detection plays a crucial role in modern agriculture. Early and accurate identification of plant diseases can prevent crop loss, reduce pesticide usage, and enhance food security. Manual inspection of plants is labor-intensive, time-consuming, and prone to human error. This motivates the use of computer vision and machine learning techniques that can detect diseases efficiently and accurately. Image-based methods provide a scalable solution, especially for large-scale farming operations.

2.1.2 Plant Disease Analysis Approaches

Several approaches have been proposed for automated plant disease detection:

- **Classical Image Processing:** Early methods rely on handcrafted features such as color, shape, and texture. Techniques like histogram analysis, edge detection, and morphological operations are applied to segment and classify diseased regions. While simple, these methods struggle with complex backgrounds and variable lighting conditions.
- **Traditional Machine Learning:** Algorithms such as Support Vector Machines (SVM), Random Forest, and k-Nearest Neighbors (kNN) classify diseases using extracted features. These approaches require careful feature selection and preprocessing but provide interpretable results.
- **Convolutional Neural Networks (CNNs):** CNNs automatically learn hierarchical features from raw images, outperforming traditional methods. Popular architectures include:
  - **VGG16/VGG19:** Deep but computationally intensive models with simple sequential layers.
  - **ResNet50/ResNet101:** Incorporates residual connections to prevent vanishing gradients in deep networks.

  - **DenseNet121:** Uses dense connectivity to improve feature reuse and gradient flow.
  - **Xception:** Depthwise separable convolutions reduce parameters while maintaining accuracy.
  - **EfficientNetB0-B1/V2:** Scalable models balancing accuracy and efficiency.
  - **NASNetLarge/Mobile:** Neural architecture search-based models optimized for accuracy or lightweight deployment.
  - **ConvNeXtTiny and MobileNetV2:** Designed for resource-limited edge devices.

- **Transfer Learning and Fine-Tuning:** Pretrained models on large datasets (ImageNet) are adapted to plant disease datasets (PlantDoc, PlantVillage). Fine-tuning allows models to learn domain-specific features, improving generalization across datasets.

- **Data Augmentation:** Techniques such as rotation, flipping, cropping, and color adjustments increase dataset diversity, reduce overfitting, and enhance model robustness.

- **Model Compression and Quantization:** INT8 and FP32 quantization reduce model size for deployment on mobile or embedded devices, enabling efficient inference with minimal accuracy loss.

2.2 Literature Survey

Recent research highlights the effectiveness of deep learning and ensemble learning in plant disease detection:

- CNN architectures achieve high accuracy for plant disease classification, but generalization across datasets is challenging due to variations in imaging conditions and disease symptoms.

- Fine-tuning pretrained networks on specific datasets improves performance, especially when there are domain differences between training and testing images.

- Lightweight models such as MobileNet and NASNetMobile enable deployment on edge devices without sacrificing too much accuracy, while larger models like NASNetLarge or EfficientNetV2L provide higher accuracy but require more resources.

- Ensemble learning techniques, such as combining predictions from multiple CNN models, can further enhance accuracy and robustness. Random Forests have also been used for combining features extracted from CNN layers.

- Quantization techniques, including INT8 and FP32, make deep learning models suitable for real-time applications on devices with limited computational capacity.

- Data preprocessing and feature engineering remain important. Normalization, resizing, and careful handling of class imbalance significantly affect model performance.

- Cross-dataset evaluation is increasingly emphasized in literature to ensure that models trained on one dataset (e.g., PlantDoc) can perform well on other datasets (e.g., PlantVillage) to test generalization capabilities.
- Studies by Hemanth et al. and Roseline et al. demonstrate that ensemble methods outperform single classifiers in detecting complex patterns in images, which is crucial for robust plant disease detection.
- Recent works highlight hybrid approaches that combine CNN-based feature extraction with ensemble methods or attention mechanisms to focus on disease-specific regions of leaves, further improving detection accuracy.

2.2.1 Challenges in Plant Disease Detection

Despite progress, several challenges remain:

1. Variability in lighting, background, and camera quality affects model performance.
2. Similarity between symptoms of different diseases can confuse models.
3. Large models require significant computational resources for training and inference.
4. Deployment on mobile and edge devices requires careful model optimization and quantization.
5. Dataset diversity and cross-network validation are crucial to ensure real-world applicability.

2.2.2 Summary

This chapter reviewed theoretical background and literature on automated plant disease detection. Key takeaways include:

- CNN-based deep learning models provide accurate and scalable classification.
- Transfer learning and fine-tuning improve model generalization across datasets.
- Ensemble methods and hybrid architectures enhance robustness and accuracy.
- Quantization and model compression are necessary for deployment in resource-constrained environments.

These insights motivate our approach: evaluating multiple CNN models, fine-tuning them for optimal performance, and analyzing trade-offs between accuracy, model size, and deployability.

Chapter 3

Proposed Approach and Algorithm

3.1 Preliminaries

In this section, we define the basic concepts, notation, and assumptions used in our approach:

- Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ denote the PlantDoc dataset, where x_i represents the input image and y_i is the corresponding disease label.
- The model $f_\theta(x)$ denotes a deep learning classifier parameterized by θ, which maps input images to predicted disease classes.
- The objective is to learn parameters θ that minimize a loss function $\mathcal{L}(y, f_\theta(x))$, typically categorical cross-entropy, over the training set.
- Data augmentation functions $A(\cdot)$ are applied to the input images to generate transformed samples, improving the generalization of the model.
- Model quantization functions $Q(\cdot)$ reduce the precision of weights and activations (e.g., from FP32 to INT8) to enable efficient deployment on resource-constrained devices.

3.2 Formulation of the Problem

The task is formulated as a multi-class image classification problem. Given an input image $x \in \mathbb{R}^{H \times W \times 3}$, the goal is to predict its disease label $y \in C$, where $C = \{c_1, c_2, ..., c_k\}$ is the set of disease classes. Formally:

$$\hat{y} = \arg\max_{c \in C} P(y = c \mid x, \theta) \tag{3.1}$$

Here, $P(y = c \mid x, \theta)$ represents the predicted probability for class c. The model is optimized to maximize the likelihood of the correct labels across the training set.

3.3 Proposed Algorithm

Our proposed approach consists of several stages aimed at maximizing accuracy and deployability:

1. **Data Preprocessing:**
   - Input images are resized to 299×299 pixels to match Xception input requirements.
   - Pixel values are normalized to [0,1] to stabilize training.
   - Data augmentation techniques such as random rotations, horizontal/vertical flips, zoom, and brightness adjustments are applied to reduce overfitting.
2. **Model Selection:**
   - Xception is chosen for its efficiency in feature extraction using depthwise separable convolutions.
   - Pretrained ImageNet weights are used to leverage learned general features and accelerate convergence.
3. **Warmup Training:**
   - Base layers of Xception are frozen, and only the top classification layers are trained for 3 epochs.
   - This allows the model to adapt the final layers to the PlantDoc dataset without disturbing pretrained features.
4. **Fine-Tuning:**
   - Last 60 layers of Xception are unfrozen to allow adaptation of high-level feature maps.
   - Learning rate is reduced using schedulers to prevent catastrophic forgetting.
   - Early stopping and ReduceLROnPlateau callbacks monitor validation metrics for efficient training.
5. **Evaluation:**
   - Metrics such as Accuracy, F1-score, Precision, Recall, ROC-AUC, and Cohen's Kappa are calculated to comprehensively evaluate model performance.
   - Confusion matrices and classification reports are generated to identify class-wise strengths and weaknesses.
6. **Quantization:**
   - INT8 and FP32 quantization techniques are applied to reduce model size and inference time.
   - This ensures the model can be deployed on mobile or edge devices without significant loss in accuracy.

3.4 Theoretical Analysis of the Algorithm

- **Convergence:** Leveraging pretrained models accelerates convergence by providing strong initial feature representations.
- **Generalization:** Data augmentation and fine-tuning improve generalization on unseen plant images, reducing overfitting.
- **Parameter Optimization:** Selective unfreezing focuses learning on relevant layers, allowing efficient adaptation to the target dataset while avoiding unnecessary weight updates.
- **Deployability:** Quantization ensures model weights require less memory and computations, making it feasible for real-world applications.
- **Scalability:** The modular design allows evaluation of other CNN architectures with minimal modification.

3.5 Algorithm Workflow

1. **Load Dataset:** Load PlantDoc images and labels.
2. **Preprocess:** Resize, normalize, and augment images.
3. **Select Model:** Initialize Xception with ImageNet weights.
4. **Warmup Training:** Train top layers for initial adaptation.
5. **Fine-Tuning:** Unfreeze last 60 layers and train with learning rate scheduling.
6. **Evaluation:** Compute metrics, generate confusion matrix, and visualize classification performance.
7. **Quantization:** Convert model to INT8 for deployment and validate accuracy.

3.6 Summary

The proposed approach combines transfer learning, fine-tuning, and data augmentation to achieve high accuracy on the PlantDoc dataset. Evaluation metrics and visualizations provide insight into class-wise performance, while quantization enables efficient deployment on edge devices, maintaining a balance between accuracy, size, and speed. This framework can be extended to other datasets and CNN architectures for broader applicability.

Chapter 4

Methodology, Implementation, Evaluation & Analysis

4.1 Tools and Platforms Used

- **Python:** Programming language for preprocessing, training, and evaluation.
- **TensorFlow & Keras:** For model creation, training, and fine-tuning.
- **Scikit-learn:** For metrics computation and data splitting.
- **Google Colab:** GPU-enabled cloud platform for model training.
- **Matplotlib & Seaborn:** For visualization of metrics, confusion matrices, and training curves.

4.2 Dataset Used

- **PlantDoc Dataset:** Contains images of multiple plant species labeled with disease types.
- Images resized to 299×299, normalized to [0,1].
- Dataset split: 80% training, 20% validation/testing.
- Class imbalance handled using weighted loss functions.

4.3 Model Training and Testing Approach

4.3.1 Initial Keras Model Evaluation by Team Members

- Each team member trained different pretrained models: Xception, VGG16/19, ResNet50V2, DenseNet121, EfficientNetB0/V2, NASNet, ConvNeXt, MobileNet.
- Metrics compared to select the best-performing model.
- Xception showed the highest baseline test accuracy.

4.3.2 Fine-tuning Xception Model

- Top layers trained first (warmup phase).
- Last 60 base layers unfrozen for fine-tuning with lower learning rate.
- EarlyStopping and ReduceLROnPlateau callbacks used.
- INT8 and FP32 quantization applied for deployment efficiency.

4.4 Metrics of Evaluation

- **Accuracy:** Overall classification correctness.
- **Precision, Recall, F1-score:** Per-class evaluation.
- **ROC-AUC:** Measure of discrimination capability.
- **Cohen's Kappa:** Agreement measure beyond chance.

4.5 Flow of Implementation

4.5.1 Individual Model Training Workflow

1. Load dataset and apply preprocessing.
2. Apply data augmentation on training images.
3. Train selected CNN model.
4. Evaluate on validation/test set.

4.5.2 Fine-tuned Xception Model Workflow

1. Warmup training: top layers only.
2. Unfreeze last 60 layers and train with reduced learning rate.
3. Monitor validation metrics and apply callbacks.
4. Apply model quantization.
5. Evaluate final model performance.

4.6 Results

4.6.1 Xception Model Summary

Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 299, 299, 3)	0
augment (Sequential)	(None, 299, 299, 3)	0
xception (Functional)	(None, 10, 10, 2048)	20,861,480
global_average_pooling2d (GlobalAveragePooling2D)	(None, 2048)	0
dropout (Dropout)	(None, 2048)	0
dense (Dense)	(None, 512)	1,049,088
batch_normalization_4 (BatchNormalization)	(None, 512)	2,048
dropout_1 (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 27)	13,851

Total params: 21,926,467 (83.64 MB)

Trainable params: 1,063,963 (4.06 MB)

Non-trainable params: 20,862,504 (79.58 MB)

Figure 4.1: Architecture and summary of the Xception model used.

4.6.2 Classification Report

Figure 4.2: Classification report showing Precision, Recall, and F1-score for each class.

4.6.3 Confusion Matrix

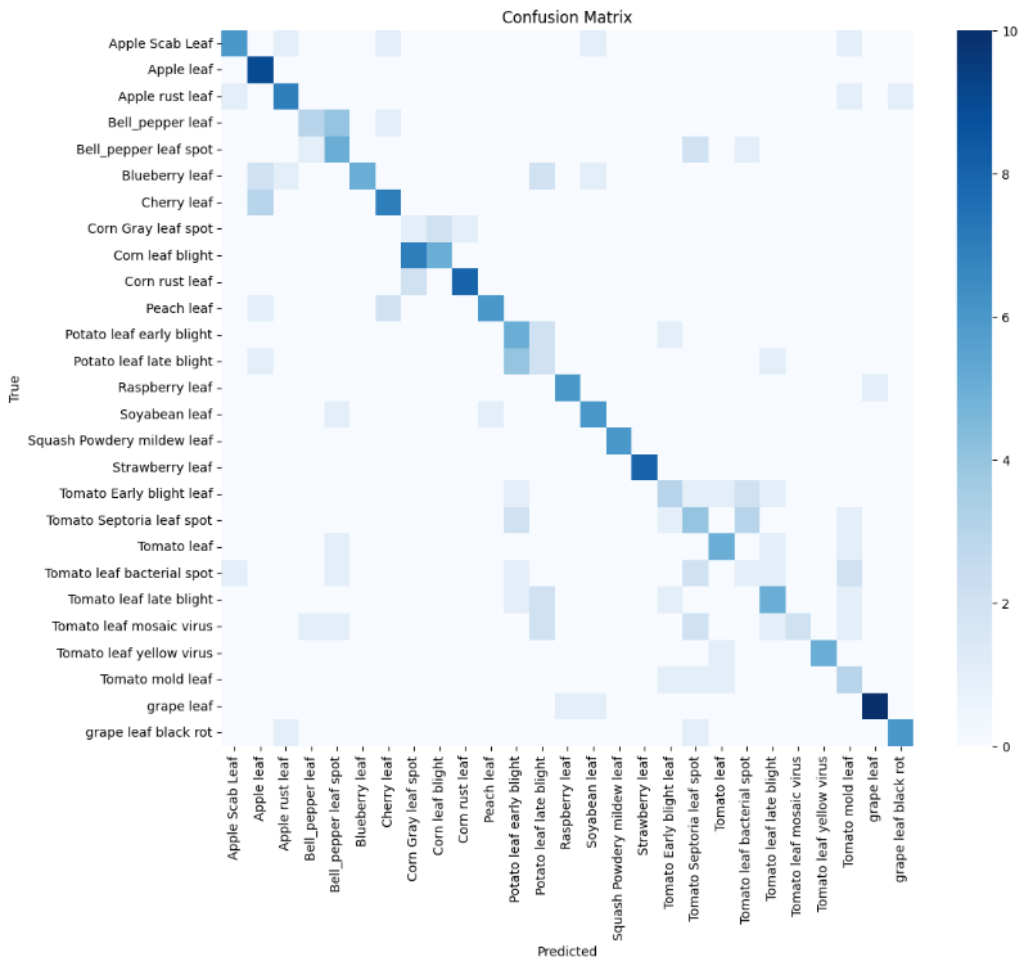

Figure 4.3: Confusion matrix for the fine-tuned Xception model on the test set.

4.6.4 Discussion and Analysis

- Fine-tuning improved accuracy and F1-score over baseline.
- Data augmentation and weighted loss helped minority classes.
- Quantization reduced model size with minimal performance loss.
- The model is suitable for deployment on edge devices with limited resources.

Chapter 5

Conclusion and Future Scope

5.1 Limitations of the Proposed Solution

While our approach achieves significant accuracy and deployability, several limitations remain:

- **High Computational Cost:** Training deep CNN models like Xception requires powerful GPUs and substantial time, which may not be accessible to all users.
- **Dependence on Dataset Quality:** Model performance is sensitive to the quality, diversity, and labeling accuracy of the PlantDoc dataset. Noisy or imbalanced data can reduce accuracy.
- **Rare Class Misclassification:** Diseases with few examples are still prone to misclassification due to limited representation in the training set.
- **Environmental Variability:** Changes in lighting, camera angles, and background can affect prediction accuracy, especially for images captured in real-field conditions.
- **Limited Real-time Evaluation:** The current study primarily evaluates offline datasets. Performance on live, real-time deployment has not been fully tested.

5.2 Impact

The proposed model offers significant practical benefits for agriculture and plant disease management:

- **Automation:** Reduces dependency on manual inspection, enabling faster and more consistent disease detection.
- **Precision Agriculture:** Supports targeted interventions such as selective pesticide application, minimizing chemical usage and reducing cost.
- **Productivity Enhancement:** Early and accurate disease detection helps prevent crop losses, improving overall yield and farm profitability.
- **Scalability:** The framework can be adapted to other crops and datasets, providing a foundation for broader plant health monitoring.

- **Educational Utility:** Serves as a case study for students and researchers interested in applying deep learning to real-world problems.

5.3 Future Scope

The proposed approach can be extended in several ways to improve performance, scalability, and real-world utility:

- **Lightweight Model Deployment:** Utilize model pruning, knowledge distillation, or mobile-friendly architectures (e.g., MobileNetV3, EfficientNet-Lite) for real-time inference on edge devices.
- **Transformer-Based Models:** Explore vision transformers (ViT) or hybrid CNN-transformer models for better feature representation and cross-domain generalization.
- **Real-Time Field Deployment:** Implement the model on drones, smartphones, or IoT devices for live monitoring and immediate disease detection in farms.
- **Dataset Expansion:** Include more plant species, disease types, and image variations to improve model robustness and generalization.
- **Multi-Modal Disease Detection:** Integrate environmental data (temperature, humidity, soil conditions) with image data to create a holistic plant health monitoring system.
- **Explainable AI:** Incorporate attention maps or saliency-based techniques to interpret model predictions, helping agronomists understand the decision-making process.
- **Predictive Analytics:** Extend the system to predict disease progression trends and recommend preventive measures before symptoms become severe.
- **Collaborative Systems:** Connect with cloud-based platforms to aggregate data from multiple farms for large-scale agricultural insights and regional disease prediction.

5.4 Concluding Remarks

This study demonstrates that deep learning models, particularly Xception with fine-tuning and quantization, can effectively identify plant diseases from images. The approach balances high accuracy with deployability, making it practical for real-world agricultural applications. Future work focusing on lightweight models, real-time deployment, and multi-modal data integration has the potential to transform precision agriculture and improve global crop management strategies.

References

PlantDoc Dataset, *PlantDoc: A Dataset for Visual Plant Disease Detection*, 2020, `https://github.com/pratikkayal/PlantDoc-Dataset`.

F. Chollet, *Xception: Deep Learning with Depthwise Separable Convolutions*, CVPR 2017.

K. Simonyan, A. Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, arXiv:1409.1556, 2015.

K. He, X. Zhang, S. Ren, J. Sun, *Deep Residual Learning for Image Recognition*, CVPR 2016.

G. Huang, Z. Liu, L. Van Der Maaten, K. Q. Weinberger, *Densely Connected Convolutional Networks*, CVPR 2017.

M. Tan, Q. Le, *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*, ICML 2019.

B. Zoph et al., *Learning Transferable Architectures for Scalable Image Recognition*, CVPR 2018.

M. Sandler et al., *MobileNetV2: Inverted Residuals and Linear Bottlenecks*, CVPR 2018.

M. Abadi et al., *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*, 2015, `https://www.tensorflow.org`.

F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, JMLR 12, pp. 2825–2830, 2011.

D. P. Hughes, M. Salathé, *An Open Access Repository of Images on Plant Health to Enable the Development of Mobile Disease Diagnostics*, arXiv:1511.08060, 2015.